

GameTrainer — Milestone M1

Borrow the Brain: Proving the Loop Can Learn

Author: Phillip • Date: July 7, 2026

Report type: milestone retrospective (M1)

Abstract

Milestone 1 (M1) replaces Milestone 0's random action-picker with a learning agent — PPO, borrowed from Stable-Baselines3 — and proves it beats the M0 baseline without changing the Gymnasium contract at all. This is the smallest possible demonstration of GameTrainer's central thesis: because every Ground speaks the same Link (the Gymnasium contract), a new Brain can be dropped into the same socket with zero changes to the environment or the loop around it. Mean reward climbed from the M0 random baseline of about 22 toward CartPole's ceiling of 500 within a 25,000-timestep training budget, comfortably clearing the 44-reward pass threshold. M1 is therefore the first proof that the borrowed brain actually learns — a result M2 will build on directly by pointing the same, unchanged brain at a Ground the project authors itself.

1. Project Context & Mental Model

GameTrainer trains an AI agent to play games, but its real subject is an **architecture**: a clean, swappable link between any game world and any AI "brain." The project follows a strict milestone order, M0 through M6, and M1 is the second step — still small, still low-risk, but the first milestone where the agent actually learns anything.

The shared vocabulary from M0 carries forward unchanged: the **Ground** (environment) produces **observations** and **rewards**; the **AI** is eyes + brain + hands; the **Link** is the Gymnasium contract (`reset()` / `step(action)`) that lets any Ground and any AI connect through the same socket; and the core loop is always **observe** → **act** → **reward** → **repeat**. The design principle of **build vs. borrow** is the throughline for M1 specifically: the project builds the Ground and the Link, but borrows the Brain — PPO, from the Stable-Baselines3 ("SB3") library.

This continues the pivot from GameTrainer's original 2025 direction (a C++ / Windows-API effort that read game memory and hand-authored rule-engine logic) to the reinforcement-learning approach adopted in early 2026: learn to play from the outside, pixels in and key-presses out, rather than reading memory or scripting rules. M0 proved the loop runs; M1 is the first milestone in that fresh foundation where something is actually being learned.

#	Goal	"Done when..."
M0 ✓	Setup	Repo + virtual environment created, libraries installed, a script runs CartPole with random actions for 100 steps without crashing.
M1 ✓	Borrow the brain	PPO trains on CartPole through our runner; average reward clearly rises vs. the random baseline.

#	Goal	"Done when..."
M2	Build our own Ground	A GridWorld env obeys the Gymnasium contract; a random agent runs, then PPO learns to reach the goal.
M3	Add the Eyes	A ViT feeds a picture of the world to PPO; the agent learns from pixels.
M4	Make it swappable	Profiles + reward calculators exist; switching environments is config-only.
M5	Add the Hands	Real key presses drive a small real game window end-to-end.
M6	(Stretch) Stardew	Stardew becomes "just another profile."

Table 1. Full M0–M6 roadmap. M0 and M1 are complete as of this report.

Timeline. M1 was built roughly June 18–27, 2026 (the implementation branch merged June 27, 2026; an internal review is dated June 22, 2026). Like M0, M1 ran entirely on CPU — no GPU is needed at this stage.

2. Milestone 1 — Borrow the Brain

Objective. Replace the random action-picker with a learning agent (PPO) and prove it beats the M0 baseline — without changing the Gymnasium contract at all.

"Done when..." PPO trains on CartPole through our runner; average reward clearly rises versus the random baseline.

2.1 Terms

- **Reinforcement Learning (RL)** — learning by trial, error, and reward: the agent tries actions, sees rewards, and adjusts to earn more over time.
- **Policy** — the agent's current strategy, a mapping from an observation to an action. "Training" means gradually improving the policy.
- **PPO (Proximal Policy Optimization)** — the specific, widely-used RL algorithm borrowed from Stable-Baselines3. The project does not write the learning math; it hands PPO an environment and tells it to learn. "Proximal" means each update is clipped to stay close to the previous policy, which keeps training stable.
- **MLP / MlpPolicy** — a Multi-Layer Perceptron, i.e. a small fully-connected neural network. Because CartPole's observation is just 4 numbers (not pixels), an MLP is the right, lightweight choice: "numbers in, action out." The pixel-based CnnPolicy is reserved for the ViT phase, M3.
- **Callback** — a hook Stable-Baselines3 runs periodically during training to do side jobs like evaluating or saving.

2.2 What We Built

- 1. PPO training script** — `scripts/train_cartpole.py`. It builds a PPO agent with `MlpPolicy`, trains it on `CartPole-v1` for 25,000 timesteps by default (configurable via `--steps`), then prints a pass/fail verdict comparing the best mean reward to the M0 baseline. A `--render` flag lets you watch the agent play during evaluation.
- 2. Two SB3 callbacks.** `EvalCallback` runs every 2,000 steps: 10 evaluation episodes on a separate evaluation environment, logging the mean reward so a learning curve can be produced and the best model saved. `CheckpointCallback` saves a model snapshot every 5,000 steps to `models/ppo_cartpole/`.
- 3. Baseline reporting** was added to `run_cartpole.py` so the M0 number the agent must beat is explicit and reproducible.
- 4. NullInput stays wired**, still a no-op — a visible reminder that swapping the decision-maker did not disturb the hands seam established in M0.

2.3 Hyperparameters

Setting	Value	Plain meaning
<code>learning_rate</code>	3e-4	How big each learning step is.
<code>n_steps</code>	2048	Experience collected before each policy update.
<code>batch_size</code>	64	Mini-batch size for each update.
<code>n_epochs</code>	10	Passes over each batch of experience.
<code>gamma</code>	0.99	Discount factor — how much future reward is worth now.
<code>gae_lambda</code>	0.95	Bias/variance trade-off when estimating “how good” an action was.
<code>clip_range</code>	0.2	PPO’s “proximal” clip — keeps each update conservative.
<code>ent_coef</code>	0.0	Extra exploration bonus (none needed for CartPole).

Table 2. PPO hyperparameters used for the M1 training run.

2.4 Experimental Method

The agent trains on one environment but is **evaluated on a separate environment**, so evaluation scores aren’t contaminated by exploration. Evaluation uses a **deterministic policy** — the agent picks its current best action rather than sampling — giving a clean measure of learned skill. The pass rule is quantitative:

Quantity	Value
M0 random baseline	22 reward / episode
Pass threshold (2× baseline)	44 reward / episode
CartPole ceiling	500 reward / episode (maximum possible)

2.5 Results

Per the milestone's own UML notes, mean reward climbed from ≈ 22 (random) toward the ceiling of 500 (trained PPO) within the 25,000-step budget, clearly clearing the 44 pass threshold. **M1 passed.**

Learning-curve detail (pending insertion): exact per-evaluation figures are reproducible from `logs/cartpole/evaluations.npz`, or a TensorBoard “mean_reward” curve via `tensorboard --logdir logs/cartpole` and are not yet transcribed into this report. The verified result stands regardless: reward rose from the ~ 22 random baseline past the 44 pass threshold toward the 500 ceiling. No intermediate step-by-step numbers are presented here to avoid stating figures that have not been directly pulled from the logs.

2.6 The Architectural Proof

This is the whole thesis in miniature. Going from a random action-picker to a learning PPO agent required **no change to the Gymnasium contract** and no change to the environment. Same Link, same Ground, a new Brain dropped into the same socket. That is exactly the swappability the entire project is built to demonstrate — proven here at the smallest possible scale.

2.7 What M1 Lends to Later

M1 proves the borrowed brain works on a borrowed Ground. That directly sets up M2, where the project builds its own Ground (GridWorld) and plugs the same, unchanged PPO brain into it. In fact, the M1 training script (PPO + MlpPolicy + EvalCallback + CheckpointCallback + pass/fail verdict) becomes the reusable template copied almost verbatim into `scripts/train_gridworld.py`.

3. Architecture at M1

Exactly one thing changes from the M0 diagram: the Agent (SB3 PPO) is connected, replacing random actions, and is trained on the same CartPole contract via two SB3 callbacks. NullInput is still wired and still a no-op. Solid arrows with hollow heads denote interface implementation; dashed arrows denote a run-time dependency.

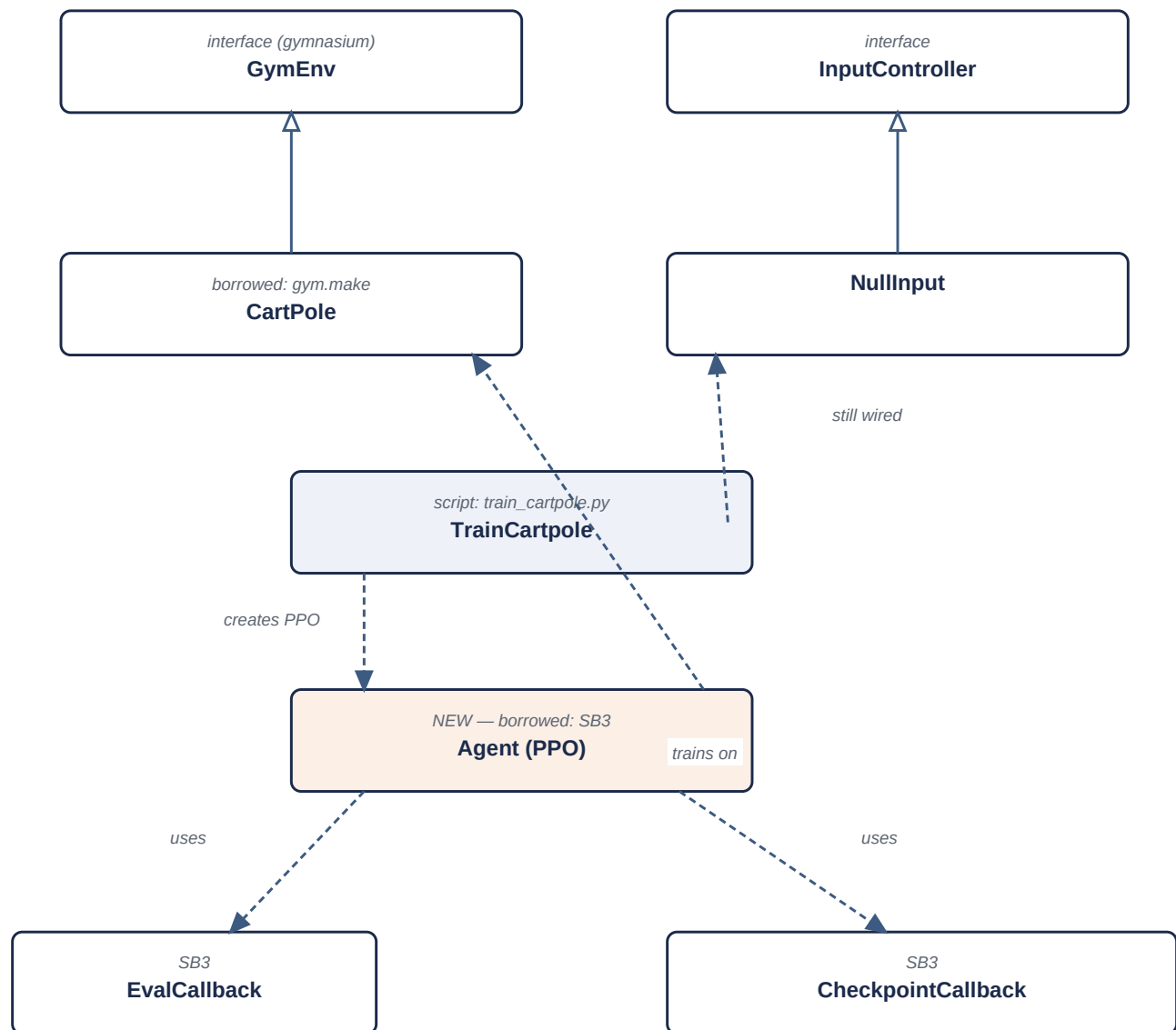


Figure 1. M1 architecture — *TrainCartpole* creates the PPO Agent, which trains against *CartPole* through the unchanged Gymnasium contract and uses two SB3 callbacks for evaluation and checkpointing.

Everything outside the new Agent node is identical to the M0 diagram. That is the point: a strict, unchanging contract between Ground and AI is what allows a smarter Brain to be substituted in without touching anything else.

4. Scope Discipline

M1 inherits M0's scope discipline and does not expand it. The same exclusions still hold, each mapped to the milestone where it will eventually arrive:

Still excluded at M1	Arrives at
No eyes / no ViT / no pixels — PPO still learns from CartPole's 4 numbers.	Vision is M3
No Ground of our own — CartPole is still borrowed.	Our first authored environment, GridWorld, is M2
No real key presses — the hands seam is still a no-op stub.	Real input is M5
No reward calculator or profile abstraction.	Config-driven swappability is M4
No GPU dependency — CPU is sufficient for a 4-number, MLP-based policy.	GPU only matters once the ViT phase begins

Keeping M1 this narrow — changing the brain and nothing else — is what makes the result legible: any change in performance can be attributed to the new PPO agent, not to some other simultaneous change.

5. Reflections & Lessons

- **A strict contract is what buys swappability.** The Brain could be replaced with zero changes to the Ground or the Link precisely because M0 never let the contract leak.
- **Evaluate separately from training.** Using a distinct evaluation environment with a deterministic policy avoids letting exploration noise masquerade as skill.
- **A 2× baseline is a defensible pass bar.** Defining “clearly rises” as a specific multiple of the random baseline, before training starts, keeps the verdict honest.
- **Change one variable at a time.** M1 touched the Brain and nothing else, which is what makes the improvement attributable to PPO rather than to some other simultaneous change.

6. Next Steps — M2 and Beyond

M2 builds the project's own Ground for the first time: a GridWorld environment that obeys the same Gymnasium contract. The same, unchanged PPO brain from M1 is pointed at it, first via a random agent to sanity-check the environment, then via PPO to learn to reach the goal. Only after that does M3 add the eyes (a ViT feeding pixels to PPO).

Appendix A — Reproduce It

```
pip install -e ".[rl]"
python scripts/run_cartpole.py
python scripts/train_cartpole.py # supports --steps, --render
tensorboard --logdir logs/cartpole
```

Appendix B — File Manifest

File	Role
scripts/train_cartpole.py	PPO training script: builds the agent, runs EvalCallback and CheckpointCallback, prints the pass/fail verdict.
scripts/run_cartpole.py	Random-action runner; now also reports the M0 baseline for comparison.
src/gametrainer/input.py	InputController interface and the NullInput no-op implementation (unchanged, still wired).
models/ppo_cartpole/	Checkpointed and best-evaluation model snapshots.
logs/cartpole/	EvalCallback logs, including evaluations.npz and TensorBoard event files.

Appendix C — Glossary

Reinforcement Learning (RL)

Learning by trial, error, and reward: the agent tries actions, sees rewards, and adjusts to earn more over time.

Policy

The agent's current strategy — a mapping from an observation to an action.

PPO (Proximal Policy Optimization)

The RL algorithm borrowed from Stable-Baselines3; clips each update to stay close to the previous policy.

MLP / MlpPolicy

A small fully-connected neural network policy, appropriate when the observation is a vector of numbers rather than an image.

Callback

A hook Stable-Baselines3 runs periodically during training, e.g. to evaluate or checkpoint the model.

EvalCallback

Runs evaluation episodes on a separate environment at a fixed interval and logs the mean reward.

CheckpointCallback

Saves a model snapshot to disk at a fixed training-step interval.

Baseline

The M0 random policy's average reward (≈ 22), used as the reference an agent must clearly beat.

Deterministic policy

An evaluation mode where the agent always takes its current best action instead of sampling, for a clean skill measurement.